JIVANA CONNECT

UI & Frontend Specification

A practical handoff for building the hackathon demo dashboard

Product promise
Every patient connected. Every follow-up accounted for.

This document consolidates the UI direction, screens, key interactions, mock-data expectations, and frontend/backend integration plan for Jivana Connect. It is intended to be shared directly with the teammate building the React frontend.

Product layer	Responsibility
Jiya voice agent (Bolna)	Conducts patient follow-up conversations and produces call results/extractions.
FastAPI backend	Owns workflow rules, validation, task status, retries, escalation decisions, and authorized actions.
MongoDB	Stores patients, appointments, tasks, calls, escalations, and event/audit records.
Web dashboard	Lets staff see what needs attention, review call outcomes, and take the next action.

Important demo rule: Use synthetic/fictitious patient information. Make mock or simulated functionality visibly clear. Do not imply that an unimplemented integration is live.

1. What the UI should communicate

Jivana Connect is an AI-powered patient engagement and care-coordination product. The dashboard should make the operational loop obvious: a follow-up is due, Jiya contacts the patient, the system extracts a structured result, the backend validates it, and the dashboard shows the next action.

Core loop

CALL → UNDERSTAND → EXTRACT → VALIDATE → UPDATE → TAKE THE NEXT ACTION

The UI is not just a list of calls. It is a staff workspace for knowing which patient follow-ups are due, which have been completed, and which require a human to act.

MVP priorities

- **Show operational clarity:** every task has a clear status, owner/department where relevant, due date, and next action.
- **Make state changes visible:** for example, an appointment task can move from “Pending” to “Rescheduling requested” and then “Awaiting staff action.”
- **Keep voice and workflow separate:** a call can finish while the follow-up task remains unresolved.
- **Prefer one complete working path:** implement appointment confirmation end-to-end first. Other workflows may be prototype/demo states if they are not fully wired.

Suggested visual direction

Element	Direction
Overall feel	Calm, trustworthy, clinical-operations workspace; clean and modern rather than consumer-chatbot-like.
Primary colour	Deep navy for navigation and headings; teal for primary actions and positive progress.
Background	White content surfaces on a very light blue-grey page background.
Status colours	Green = complete/confirmed; amber = pending/needs attention; red = urgent/escalation; grey = inactive/unknown.
Layout	Persistent left sidebar, top header with page title and demo environment indicator, content area with cards and tables.
Density	Readable at a glance. Use concise labels, clear spacing, and tables with visible status badges.

These are implementation recommendations for consistency; they are not a claim that a final visual design already exists.

2. Recommended frontend stack and structure

Area	Recommendation
Framework	React + TypeScript + Vite
Styling	Tailwind CSS
UI components	shadcn/ui or a consistent equivalent component set
Navigation	React Router
Data layer	A small API client/service layer; keep mock data behind the same interface so it can be swapped for real API calls.
Charts	Optional. Do not spend hackathon time on decorative charts before task/call workflows work.

Suggested app shell

- **Left sidebar:** Overview, Patients, Tasks & Follow-ups, Appointments, Calls, Escalations, Settings.
- **Top bar:** current page title, optional search, environment badge such as “DEMO DATA”, and profile/menu placeholder.
- **Main content:** page-specific cards, filters, tables, detail panels, and actions.
- **Reusable components:** StatusBadge, SummaryCard, DataTable, EmptyState, LoadingState, ErrorBanner, ConfirmDialog, PatientSummary, Timeline/EventList.

Suggested routes

Route	Purpose
/	Overview dashboard
/patients	Patient directory
/patients/:patientId	Patient detail and recent activity
/tasks	Tasks and follow-ups queue
/tasks/:taskId	Task detail, status, and actions
/appointments	Appointment list and confirmation/rescheduling status
/calls	Call history
/calls/:callId	Call details, summary, extracted fields, and timeline
/escalations	Open escalations requiring staff attention
/settings	Demo/configuration information; do not imply unsupported settings are live.

Authentication

If authentication is not implemented, use a clearly labelled demo-mode entry or omit the login screen. Do not create a fake login that appears to provide real security. If role-based permissions are represented, mark them as prototype-only unless enforced by the backend.

3. Screen specification: Overview dashboard

Goal: let a staff member answer in a few seconds: What needs attention today? What did Jiya complete? What requires my intervention?

Top summary cards

Card	Meaning / behavior
Patients	Number of synthetic patients in the demo dataset.
Follow-ups due	Count of tasks due now or overdue; define the rule consistently.
Calls completed	Completed calls in the selected period. Do not count failed/no-answer attempts as completed conversations.
Open escalations	Escalations that still require staff review or action.

Main sections

- **Needs attention:** prioritized list of overdue tasks, rescheduling requests, patient concerns, and callback requests.
- **Recent calls:** patient, call purpose, date/time, call outcome, related task, and a link to call details.
- **Follow-up queue:** the next due tasks, including task type, due time, status, and a clear action button.
- **Recent activity:** short event timeline such as “Call completed”, “Rescheduling requested”, “Escalation created”, or “Staff action recorded”.

Primary interactions

- Clicking a task opens its detail view; clicking a call opens its transcript/summary and extracted outcome.
- A staff member can start a call only when the task is eligible and a permitted recipient/contact exists.
- After a call webhook is processed, refresh or update the relevant task, call, dashboard counts, and activity feed.
- Use loading, empty, and error states. Avoid showing success until the backend confirms the action.

Recommended dashboard layout

Header: “Overview” + “DEMO DATA” badge. Row 1: four summary cards. Row 2: a wider “Needs attention” table on the left and “Recent calls” on the right. Row 3: follow-up queue/activity feed. On small screens, stack sections vertically.

4. Screen specifications: core pages

Patients

- Table columns: patient name or demo ID, age (only if needed for the demo), next appointment, open follow-ups, last contact, and overall attention status.
- Search/filter by name or ID and status. Do not expose unnecessary sensitive information in the list.
- Patient detail should show a concise summary, appointments, open tasks, recent calls, and a chronological activity timeline.

Tasks & Follow-ups

- This is the primary operational page. Include filters for task type, status, due date, and priority.
- Suggested columns: task, patient, due date/time, status, last attempt, and next action.
- Task detail should show purpose, safe call context, attempts, latest result, extracted structured fields, and an action history.
- Main button may be “Start Jiya call” only if the backend says the task is eligible. Show disabled/loading/error states.

Appointments

- Show appointment date/time, department or clinician if available in the synthetic data, confirmation state, and related follow-up task.
- A patient saying they need to reschedule must produce “Rescheduling requested” or an equivalent request state—not “Appointment rescheduled” unless the booking system actually confirms the change.
- Make the next step clear: staff follow-up, booking-system action, or awaiting confirmation.

Calls

- Table columns: call ID, patient, purpose, started/completed time, call status, outcome, and linked task.
- Call detail: call status, short summary, transcript if available and appropriate, extracted fields, backend processing result, and related task/escalation.
- Keep call status separate from task status. Example call status = “Completed”; task status = “Awaiting staff action”.

Escalations

- Show only items needing human review/action, with severity/priority, reason, patient, source call/task, created time, and current status.
- Detail view should show the patient’s reported concern in neutral language, the relevant call summary, the routing destination, and a staff action history.
- Available actions should be limited to implemented actions, such as acknowledge, assign, add note, or mark resolved.

Settings

Keep this lightweight for the hackathon. It can show environment, configured workflow types, and integration status. Clearly label placeholders and avoid implying that configuration controls are operational when they are not.

5. Workflows the UI must support

Workflow	UI state/result	Important rule
Appointment confirmation	Confirmed, no answer, declined, or rescheduling requested; task/call history updated.	Do not mark appointment changed unless the booking system confirms it.
Test/report reminder	Patient-reported test completion and report-received state displayed separately.	“Test completed” is not the same as “report received” or “clinician reviewed”.
Post-consultation/procedure check-in	Routine response or concern requiring staff review.	Jiya must not diagnose, prescribe, interpret results, or make clinical decisions.
Clinician-assigned task reminder	Task response captured and follow-up status updated.	Only use approved reminder wording and the actual task details.
Callback request	Requested callback window captured and displayed for staff/scheduler.	Backend validates allowed calling hours and creates the callback task.

One agent, multiple workflow configurations

Use one voice agent—Jiya—for the demo. The backend passes a specific task/workflow context to the call. The dashboard should display the call purpose and workflow so staff can tell why the patient was contacted.

Safety and clinical boundaries

- Jiya can capture what a patient reports and route a concern according to an approved escalation rule.
- Jiya must not diagnose, prescribe, interpret test results, or decide clinical urgency beyond explicitly approved routing rules.
- A patient-reported status is not proof that a test/report has been received or reviewed by a clinician.
- Escalations should remain visible until a staff member or authorized system records the next step.

6. Data and status conventions

Use controlled status values shared by frontend and backend. Do not let the UI invent status strings independently.

Entity	Suggested statuses / fields
Call	queued, initiating, in_progress, completed, no_answer, failed, cancelled
Task	pending, due, in_progress, awaiting_staff_action, completed, cancelled, failed
Appointment follow-up outcome	confirmed, rescheduling_requested, declined, no_answer, unable_to_confirm
Escalation	open, acknowledged, assigned, resolved, dismissed (only if implemented)
Test/report tracking	test_completion_reported, report_received, clinician_reviewed — store as separate fields/states, not one combined boolean.

Example state transition

Before: Appointment confirmation task = Pending.
Call result: Patient asks to reschedule.
After: Call = Completed; extracted outcome = Rescheduling requested; Task = Awaiting staff action; dashboard attention count updates; appointment itself remains unchanged until a real booking action succeeds.

Suggested record types

- **Patient:** patient_id, display_name, safe demo contact reference, created_at.
- **Appointment:** appointment_id, patient_id, scheduled_at, department/clinician if needed, booking_status.
- **Task:** task_id, patient_id, appointment_id if relevant, task_type, due_at, status, priority, attempt_count, next_action.
- **Call:** call_id, task_id, patient_id, provider_call_id, status, outcome, summary, transcript reference if available, timestamps.
- **Escalation:** escalation_id, patient_id, call_id/task_id, reason, priority, status, assigned_to, timestamps.
- **Event/audit record:** event_id, entity_type, entity_id, event_type, actor/system source, timestamp, minimal metadata.

The backend should remain the source of truth. The frontend displays backend results; it should not directly write to MongoDB or make clinical/workflow decisions.

7. Suggested API contract

Use these as proposed endpoints for coordinating the frontend and FastAPI backend. Adjust names only by agreement between teammates.

Method + endpoint	Purpose
GET /api/dashboard/summary	Return counts and dashboard summary.
GET /api/patients	List/search patients.
GET /api/patients/{patient_id}	Patient detail and related activity.
GET /api/appointments	List appointments and their current booking status.
GET /api/tasks	List/filter follow-up tasks.
POST /api/tasks	Create a task (if this is part of the demo).
GET /api/tasks/{task_id}	Get task detail and history.
POST /api/tasks/{task_id}/start-call	Ask backend to validate eligibility and initiate Jiya call.
GET /api/calls	List calls.
GET /api/calls/{call_id}	Call detail and extracted result.
GET /api/escalations	List escalations.
PATCH /api/escalations/{escalation_id}	Update an allowed escalation field/action.
POST /api/webhooks/bolna	Backend endpoint for call lifecycle/result webhooks; not called directly by the browser.

Integration contract

- Frontend calls FastAPI only. Never put Bolna secrets or database credentials in browser code.
- The backend validates the task, contact permission, calling window, and duplicate/in-progress state before starting a call.
- Webhook handling should validate authenticity where supported and be idempotent so a repeated event does not duplicate task updates or escalations.
- After starting a call, return a clear accepted/in-progress response and let the UI poll/refetch or receive updates through a supported mechanism.
- Show API errors in plain language, preserve the task state, and let staff retry only when the backend says retry is allowed.

8. Mock data and integration strategy

The frontend teammate can work in parallel with the backend by starting with mock data that matches the agreed schemas and statuses.

Seed a small, coherent demo dataset

- Use approximately 6–10 synthetic patients, a few appointments, 8–12 follow-up tasks, several completed/no-answer calls, and 1–2 open escalations.
- Make the primary appointment-confirmation scenario internally consistent across patient, appointment, task, call, and activity timeline.
- Include one task whose result is “Rescheduling requested” and whose next action is staff follow-up.
- Include a test/report example that distinguishes test completion reported, report received, and clinician reviewed.
- Include one post-procedure concern routed to a human-review queue using a prewritten, non-diagnostic scenario.

Mock-first implementation pattern

Create a single API/service interface (for example, patientsApi, tasksApi, callsApi). Initially these functions return local mock data. When FastAPI endpoints are ready, replace the implementation without rewriting page components.

Example task object (illustrative)

```
{ "task_id": "task_demo_001", "patient_id": "patient_demo_001", "task_type":
"appointment_confirmation", "due_at": "2026-10-30T10:00:00+05:30", "status": "pending", "priority":
"normal", "attempt_count": 0, "next_action": "Start confirmation call" }
```

Demo labels

- Use a persistent “DEMO DATA” or “SIMULATED” badge anywhere the UI uses seeded/mock information.
- If the call is a real Jiya/Bolna call, label the voice interaction as live while still keeping patient data synthetic.
- Do not display a fake success toast for an action that did not reach the backend.
- If transcript, analytics, or integration status is mocked, label it as a prototype example.

9. What to show in the five-minute demo

Time	What to show
0:00–0:25	Problem: missed follow-ups and administrative workload.
0:25–0:55	Overview dashboard, summary cards, and “Needs attention” queue.
0:55–1:20	Open a pending appointment-confirmation task and show its context.
1:20–2:15	Run the real Jiya appointment-confirmation call, if working.
2:15–3:00	Show call outcome/extracted fields and the task/dashboard update.
3:00–3:35	Show the test/report reminder scenario; label simulated parts clearly.
3:35–4:10	Show a post-procedure concern routed to human review; no diagnosis or treatment advice.
4:10–4:40	Brief architecture: Bolna/Jiya, FastAPI workflow logic, MongoDB, scheduler/webhooks, dashboard.
4:40–5:00	Close with the value proposition: every patient connected, every follow-up accounted for.

Recording recommendation

A clean screen recording with the actual Jiya call audio and dashboard is enough; there is no need to film yourself or another person. Capture the dashboard before the call, the call itself, and the visible result afterward. Only claim what the recording demonstrates.

10. Frontend handoff checklist

- Agree with backend teammate on field names, status enums, timestamps, and endpoint response shapes before integration.
- Build the shared app shell and sidebar first.
- Implement Overview and Tasks pages before optional pages or decorative charts.
- Make one complete appointment-confirmation flow easy to demo.
- Ensure call completion and task completion are represented separately.
- Add loading, empty, error, disabled, and success states.
- Use consistent status badges and a clear next action on every task/detail view.
- Use synthetic data and persistent demo/prototype labels.
- Verify that rescheduling requests do not falsely change the appointment booking status.
- Test the whole path: task selected → backend starts call → call completes → webhook processed → structured outcome validated → task/dashboard refreshed.

Definition of done for the UI MVP

A teammate unfamiliar with the code can open the app, understand what needs attention, start the eligible demo call, see the call outcome, and identify the next action without needing a verbal explanation from the builder.

Scope note

This is a consolidated implementation handoff based on the previously discussed Jivana Connect concept and demo plan. It defines recommended UI behavior and integration contracts; it does not claim that every listed screen, endpoint, or workflow has already been implemented.